



MEMORIA TÉCNICA DEL PROYECTO

Plataforma de Descubrimiento Local en Euskadi

EQUIPO 4 — MÁS ALLÁ DEL GUGGEN

Ane Jauregui
Frank Rocha
Darío Arenaza
Jonathan Vélez

React 19 · FastAPI · PostgreSQL + PostGIS · Docker

© 2026 Aupa! — Viviendo Euskadi como un local

Contenido

1. Introducción	3
1.1 Resumen del Proyecto	3
1.2 Motivación	3
1.3 Arquitectura General	3
2. Flujos de Comunicación	4
2.1 Topología de Comunicación	4
2.2 Protocolo de Comunicación	4
2.3 Endpoints Principales	5
3. Objetivos	5
3.1 Objetivos Generales	5
3.2 Objetivos Específicos	5
4. Análisis de Requisitos	6
4.1 Requisitos Funcionales	6
4.2 Requisitos No Funcionales	6
5. Arquitectura y Diseño	7
5.1 Stack Tecnológico	7
5.2 Endpoints Principales	7
5.3 Modelo de Datos	8
5.4 Servicios de Datos	8
6.1 Flujo General de la Aplicación	8
.....	9
6.2 Flujo 1 — Registro e Inicio de Sesión	10
6.3 Flujo 2 — Descubrimiento y Guardado de un Lugar	11
6.4 Flujo 3 — Alta y Gestión de Negocio (Local Partner)	12
7. Detalles de Implementación	14
7.1 Motor de Recomendaciones Geoespacial	14
7.2 Portal Local Partner	14
7.3 Sistema de Internacionalización	14
8. Estructura del Proyecto	15
9. Manual de Usuario	16
9.1 Acceso y Navegación	16
9.2 Cómo Descubrir un Lugar	16
10. Retos y Soluciones	17
11. Conclusiones y Trabajo Futuro	18
11.1 Conclusiones	18
11.2 Líneas de Trabajo Futuras	18

1. Introducción

1.1 Resumen del Proyecto

Aupa es una aplicación web de tipo Single Page Application (SPA) de carácter móvil-first, diseñada para conectar a los visitantes y residentes de Bilbao con la oferta cultural, gastronómica y de ocio de la ciudad. La plataforma ofrece recomendaciones personalizadas de lugares, experiencias, eventos y establecimientos en función de la ubicación geográfica del usuario y sus preferencias individuales.

El sistema contempla tres perfiles de usuario diferenciados:

- Usuario final: Descubre, guarda y valora experiencias y lugares cerca de él.
- Local Partner: Negocios y establecimientos locales que gestionan su ficha y eventos.
- Administrador: Gestión global de usuarios, locales, categorías y contenido.

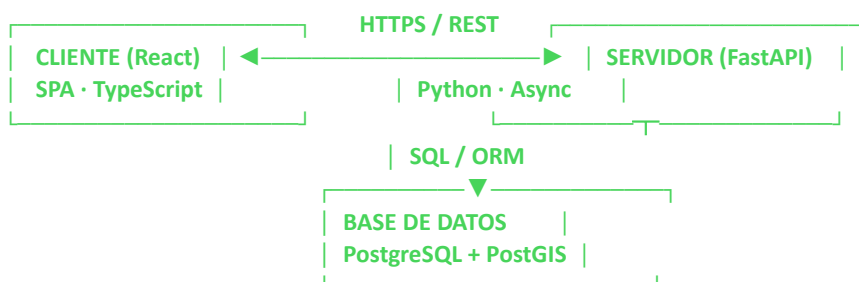
1.2 Motivación

La oferta de ocio y cultura en una ciudad como Bilbao se encuentra habitualmente dispersa entre múltiples plataformas institucionales, redes sociales y portales de turismo. Esta fragmentación dificulta al visitante la planificación de su tiempo de manera eficiente e impide descubrir el tejido local auténtico de la ciudad.

Aupa nace con el objetivo de centralizar esta información, enriquecerla con datos geoespaciales y meteorológicos, y presentarla en una experiencia de usuario cuidada, accesible y personalizable.

1.3 Arquitectura General

La aplicación sigue una arquitectura Cliente–Servidor desacoplada, compuesta por tres capas claramente diferenciadas:

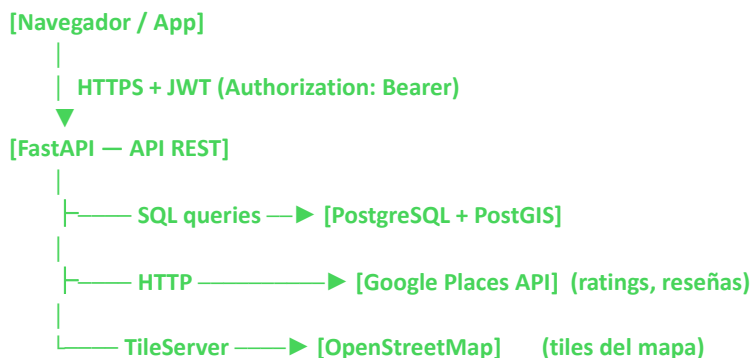


Justificación del enfoque:

- SPA (Single Page Application) con React + TypeScript: Permite una experiencia de usuario fluida en dispositivos móviles, sin recarga de página, con navegación instantánea entre vistas. TypeScript aporta seguridad de tipos y mantenibilidad al crecer el proyecto.
 - FastAPI como backend: Framework asíncrono de alto rendimiento en Python, ideal para exponer APIs REST con validación automática de esquemas (via Pydantic) y documentación interactiva integrada (Swagger/OpenAPI).
 - PostgreSQL + PostGIS: La extensión PostGIS habilita consultas geoespaciales nativas (distancia, radio de búsqueda, coordenadas), fundamentales para el motor de recomendaciones por proximidad.
 - Docker Compose: El entorno completo se orquesta mediante contenedores, garantizando consistencia entre desarrollo y producción y simplificando el despliegue.

2. Flujos de Comunicación

2.1 Topología de Comunicación



2.2 Protocolo de Comunicación

- Protocolo: HTTP/1.1 sobre HTTPS (TLS en producción).
 - Estilo de API: REST con respuestas en formato JSON.
- Autenticación: Tokens JWT (JSON Web Token) incluidos en la cabecera Authorization: Bearer <token> en todas las peticiones a rutas protegidas.
 - Mapa interactivo: El cliente consume directamente los tiles del mapa de OpenStreetMap vía la librería Leaflet.js, sin intermediar el backend.

2.3 Endpoints Principales

Método	Endpoint	Descripción
POST	/auth/register	Registro de usuario
POST	/auth/login	Autenticación y emisión de JWT
GET	/recommendations/nearest	Lugares más próximos a coordenadas dadas
GET	/recommendations/by-category	Lugares filtrados por categoría
GET	/places/{id}	Detalle completo de un lugar
POST	/saved	Guardar un lugar
GET	/local-partner	Datos del negocio local autenticado
POST	/local-partner/events	Crear un evento desde el portal Local Partner

3. Objetivos

3.1 Objetivos Generales

- Desarrollar una interfaz moderna y responsiva para el descubrimiento de lugares y experiencias locales.
- Integrar un motor de recomendaciones basado en geolocalización y preferencias de usuario.
- Proporcionar un portal de gestión independiente para negocios locales (Local Partner).

3.2 Objetivos Específicos

- Implementar consultas geoespaciales sobre PostgreSQL/PostGIS para calcular la proximidad de lugares en tiempo real.
- Integrar la API de Open-Meteo para mostrar el estado del tiempo en el punto de ubicación del usuario.
- Asegurar la accesibilidad mediante soporte multi-idioma (Castellano, Euskera, Inglés).
- Garantizar la seguridad del sistema mediante autenticación con JWT y encriptación de contraseñas con bcrypt.
- Orquestar el entorno completo (cliente, servidor y base de datos) mediante Docker Compose.

4. Análisis de Requisitos

4.1 Requisitos Funcionales

ID	Nombre	Descripción
RF1	Recomendaciones por Proximidad	El sistema mostrará lugares y experiencias ordenados por distancia al usuario.
RF2	Filtrado por Categoría	Los usuarios podrán filtrar resultados por tipo (gastronomía, cultura, naturaleza, ocio nocturno, etc.).
RF3	Detalle del Lugar	Cada lugar mostrará información completa: descripción, puntuación, reseñas, distancia y mapa.
RF4	Gestión de Guardados	Los usuarios autenticados podrán guardar y eliminar lugares de su lista personal.
RF5	Información Meteorológica	La aplicación mostrará el tiempo actual en la ubicación del usuario al inicio.
RF6	Portal Local Partner	Los negocios podrán registrarse, gestionar su ficha y publicar eventos propios.
RF7	Panel de Administración	El administrador gestionará usuarios, locales, categorías y contenido de la plataforma.
RF8	Internacionalización	La interfaz estará disponible en tres idiomas seleccionables por el usuario.

4.2 Requisitos No Funcionales

ID	Nombre	Descripción
RNF1	Rendimiento	Las consultas de recomendaciones deben resolverse en tiempos reducidos gracias a índices geoespaciales.
RNF2	Usabilidad	Diseño mobile-first adaptado a dispositivos móviles y escritorio con experiencia coherente en ambos.
RNF3	Seguridad	Comunicaciones cifradas mediante HTTPS, contraseñas hasheadas y rutas protegidas por JWT.
RNF4	Mantenibilidad	Código modular y tipado con TypeScript, siguiendo las mejores prácticas de React y separación por capas.
RNF5	Portabilidad	Entorno reproducible mediante Docker Compose, ejecutable en cualquier máquina sin configuración manual.

5. Arquitectura y Diseño

5.1 Stack Tecnológico

Tecnología / Capa	Detalle
Entorno de Desarrollo	Node.js y npm
Framework Principal (Frontend)	React 19 (Hooks, Context API) con TypeScript
Bundler	Vite 8
Enrutado	React Router 6
Estilos	Vanilla CSS con CSS Variables, Flexbox y Grid
Framework Backend	FastAPI (Python) con Uvicorn
Base de Datos	PostgreSQL 15 + extensión PostGIS
Autenticación	JWT (JSON Web Tokens) + bcrypt
API Meteorológica	Open-Meteo
Cartografía	Leaflet.js + OpenStreetMap
Orquestación	Docker + Docker Compose
Herramientas	GitHub, Jira

5.2 Endpoints Principales

Método	Endpoint	Descripción
POST	/auth/register	Registro de usuario
POST	/auth/login	Autenticación y emisión de JWT
GET	/recommendations/nearest	Lugares más próximos a coordenadas dadas
GET	/recommendations/by-category	Lugares filtrados por categoría
GET	/places/{id}	Detalle completo de un lugar
POST	/saved	Guardar un lugar
GET	/local-partner	Datos del negocio local autenticado
POST	/local-partner/events	Crear un evento desde el portal Local Partner

5.3 Modelo de Datos

El sistema utiliza modelos tipados (interfaces TypeScript en el cliente, esquemas Pydantic en el servidor) para normalizar la comunicación entre capas:

- Recommendation: estructura los datos de un lugar (id, name, description, category, latitude, longitude, distance_from_user, google_rating, reviews, address).
- User: modela al usuario autenticado (id, name, email, role, language, preferences).
- Event / Experience: representa los eventos publicados por un Local Partner (id, title, description, date, category, localId).
- APIWeather: modela la respuesta de Open-Meteo para su renderizado en la interfaz (temperature, unit, weatherCode).

5.4 Servicios de Datos

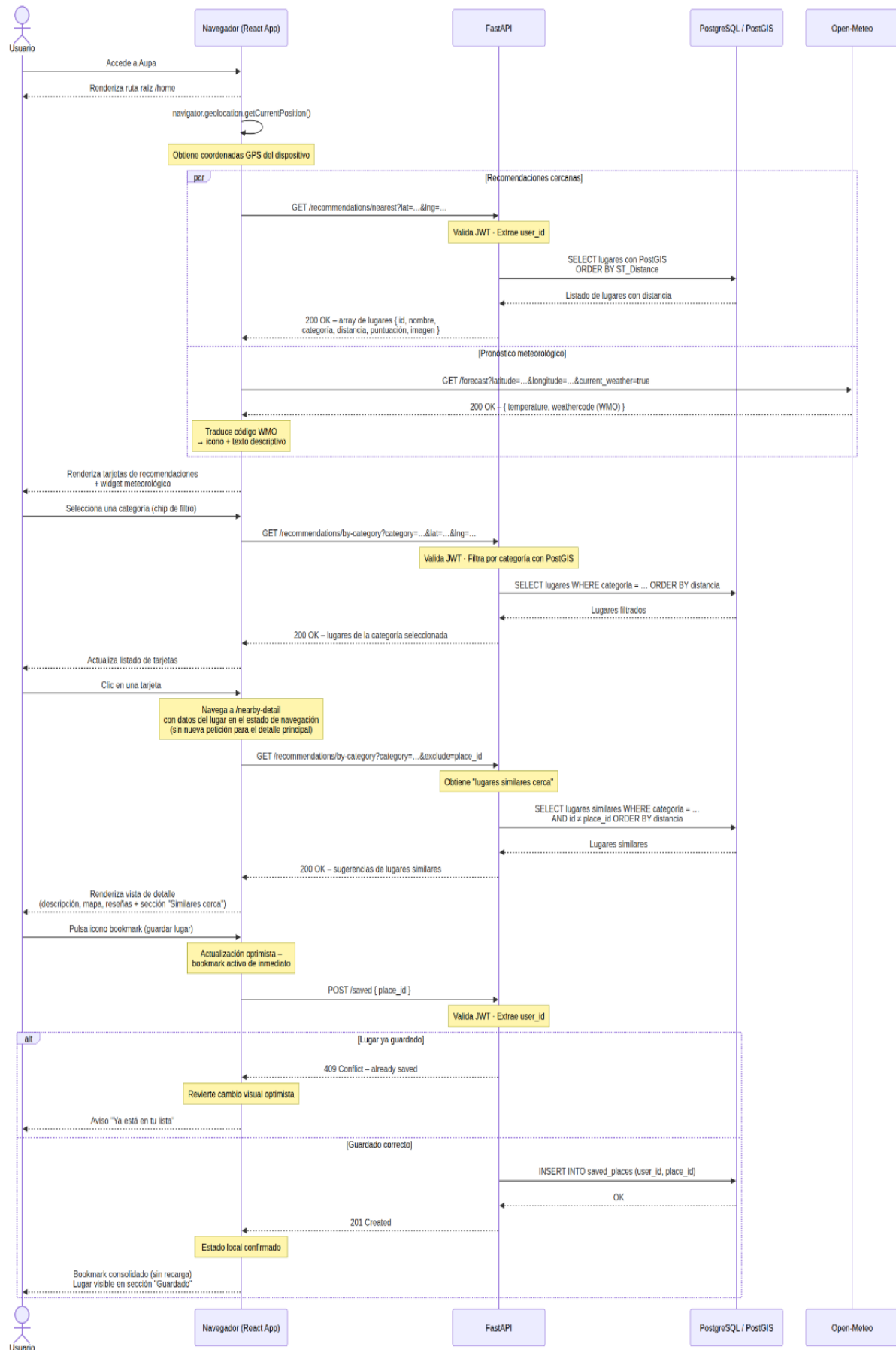
- fastapiService: comunicación con todos los endpoints del backend (recomendaciones, autenticación, guardados).
- weatherService: consulta a la API de Open-Meteo para obtener el clima actual por coordenadas.
- locationService: gestión de la geolocalización del dispositivo con fallback a coordenadas de Bilbao.
- localStorageService: persistencia auxiliar de preferencias de idioma y sesión de usuario.

6. Diagramas de Flujo

Los siguientes diagramas describen los flujos principales de la aplicación, modelados como diagramas de secuencia con Mermaid.js.

6.1 Flujo General de la Aplicación

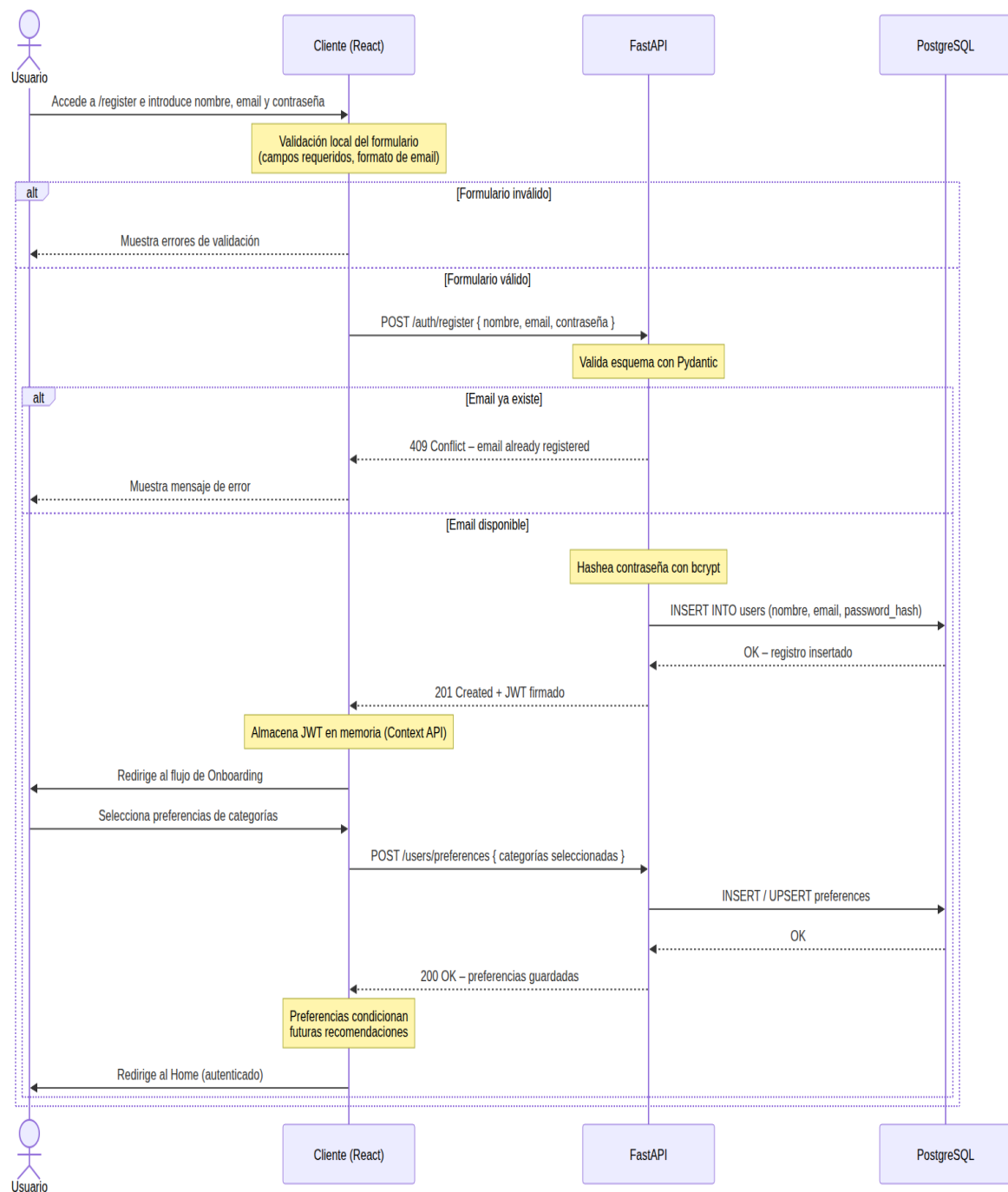
Muestra el ciclo completo de interacción del usuario: acceso inicial, geolocalización, obtención paralela de recomendaciones y pronóstico meteorológico, filtrado por categoría, navegación al detalle y guardado de un lugar.



6.2 Flujo 1 — Registro e Inicio de Sesión

Describe el proceso completo desde que un usuario nuevo se registra hasta que accede a la app con su sesión activa y sus preferencias guardadas.

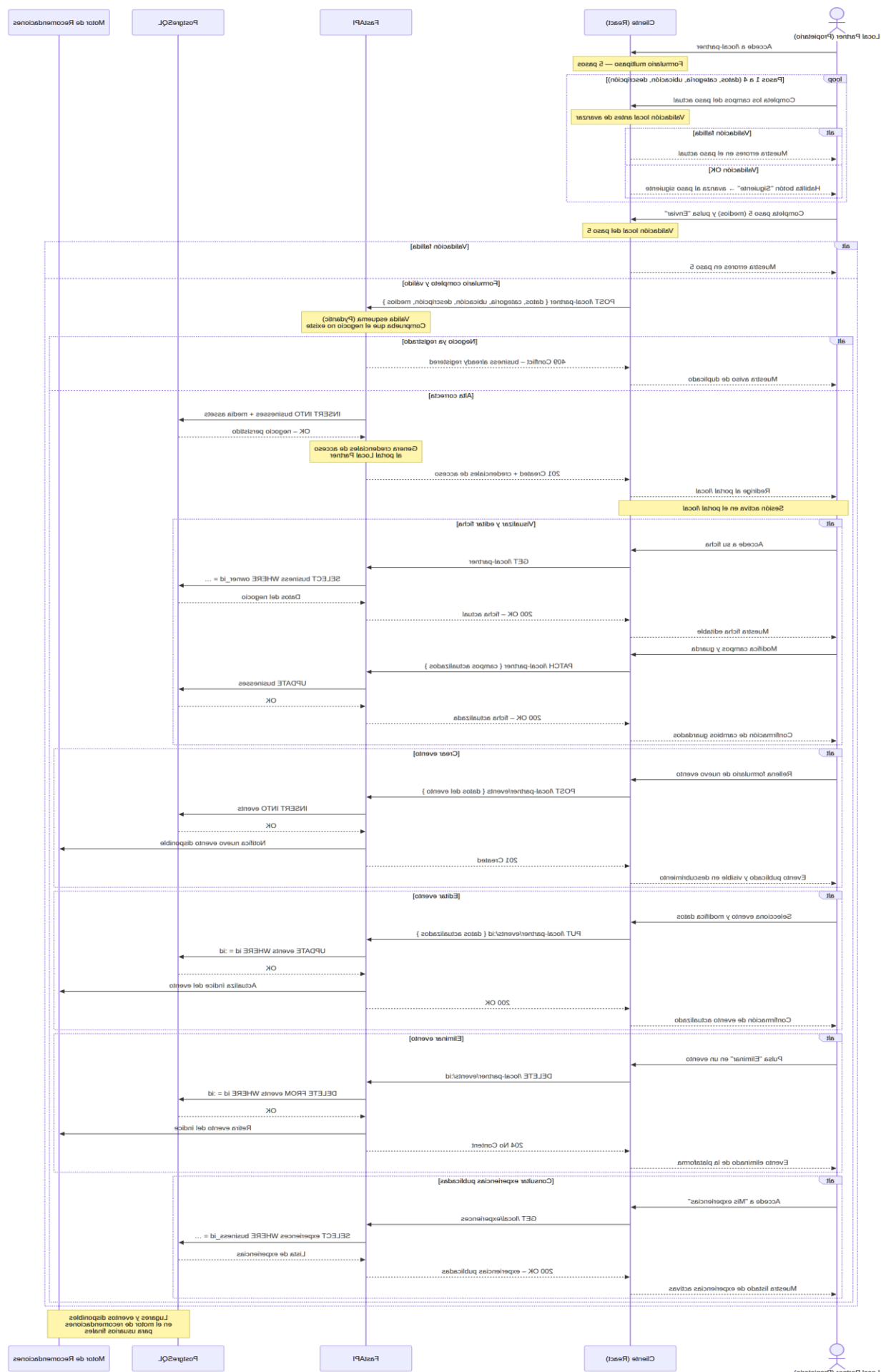
Actores: *Usuario final · Cliente React · FastAPI · PostgreSQL.*



6.4 Flujo 3 — Alta y Gestión de Negocio (Local Partner)

Describe el formulario multipaso de registro de negocio y todas las operaciones de gestión disponibles desde el portal Local Partner: edición de ficha, CRUD de eventos y consulta de experiencias.

Actores: Propietario · Cliente React · FastAPI · PostgreSQL · Motor de Recomendaciones.



7. Detalles de Implementación

7.1 Motor de Recomendaciones Geoespacial

El componente Nearby centraliza la lógica de descubrimiento. Gestiona estados para coordinar los filtros de categoría, la búsqueda por texto y la paginación de resultados. Utiliza `useEffect` para reaccionar a cambios en el filtro activo y relanzar la petición al backend.

7.2 Portal Local Partner

El formulario de registro de negocio está estructurado en cinco pasos secuenciales, cada uno con validación local antes de avanzar. El componente `PartnerLayout` gestiona el estado global del formulario y coordina la navegación entre pasos.

7.3 Sistema de Internacionalización

La aplicación utiliza un sistema de diccionarios JSON por idioma, centralizado en la función `getAppCopy(language)`. La detección del idioma preferido se realiza a partir del perfil del usuario autenticado, permitiéndole cambiarlo manualmente desde su configuración de perfil.

8. Estructura del Proyecto

Ruta	Descripción
client/public/	Recursos estáticos (imágenes, favicon, iconos)
client/src/components/	Componentes UI reutilizables (ExperienceCard, SearchBar, NearbyMap...)
client/src/pages/	Vistas principales: Home, Nearby, NearbyDetail, Saved, Experiences, Profile
client/src/services/	Lógica de APIs (fastapiService, weatherService, locationService)
client/src/utlis/	Funciones auxiliares (formatDistance, generateRandomScore, getCategoryImage)
client/src/context/	Contextos globales de React (AuthContext)
client/src/i18n/	Diccionarios de traducción y función getAppCopy
client/src/assets/styles/	Hoja de estilos global con CSS Variables, Flexbox y Grid
client/src/routes/router.tsx	Configuración de rutas y guardas de autenticación
server/	Código fuente de FastAPI (modelos Pydantic, rutas, lógica de negocio)
docker-compose.yml	Orquestación de servicios: cliente, servidor y base de datos
vite.config.ts	Configuración del entorno de desarrollo del cliente

9. Manual de Usuario

9.1 Acceso y Navegación

Al entrar en Aupa, el usuario es recibido por la pantalla de inicio con recomendaciones personalizadas y el tiempo actual en su ubicación. La barra de navegación permite acceder a:

- Inicio (/home): recomendaciones personalizadas y favoritos locales.
- Cerca de ti (/nearby): lugares ordenados por proximidad con mapa interactivo.
- Experiencias (/experiences): catálogo completo de actividades y eventos.
- Guardado (/saved): lista personal de lugares y experiencias guardados.
- Perfil (/profile): gestión de cuenta, preferencias e idioma.

9.2 Cómo Descubrir un Lugar

1. Acceda a la sección "Cerca de ti" desde la barra de navegación.
2. Conceda permiso de geolocalización o use la ubicación por defecto (Bilbao).
3. (Opcional) Seleccione una categoría en los filtros horizontales (Gastronomía, Cultura, Naturaleza...).
4. (Opcional) Escriba en la barra de búsqueda para filtrar por nombre o descripción.
5. Los resultados aparecerán en tarjetas ordenadas por distancia, con puntuación y mapa actualizado.
6. Haga clic en una tarjeta para ver el detalle completo: descripción, reseñas y lugares similares.
7. Pulse el icono de marcador para guardar el lugar en su lista personal.

10. Retos y Soluciones

Reto	Solución adoptada
Recomendaciones por proximidad en tiempo real	Integración de PostGIS en PostgreSQL para ejecutar consultas geoespaciales nativas (ST_Distance, ST_DWithin), evitando cálculos de distancia en capa de aplicación.
Gestión del estado de geolocalización	Sistema de fallback con TEST_LOCATION (coordenadas de Bilbao) para entornos donde el usuario deniega el permiso de ubicación.
Scroll y layout en móvil vs. escritorio	Arquitectura CSS con 100dvh y contenedores de scroll interno (overflow-y: auto) para móvil, complementados con media queries que liberan el scroll en escritorio.
Puntuaciones sin datos suficientes	Función generateRandomScore(seed) — determinista basada en el nombre/ID del lugar — que genera una puntuación coherente y reproducible.
Datos HTML en descripciones de la API	Uso de dangerouslySetInnerHTML de React, aplicado únicamente a contenido proveniente de fuentes controladas del backend.
Orquestación multi-servicio en desarrollo	docker-compose.yml que levanta simultáneamente el cliente (Vite), el servidor (FastAPI/Uvicorn) y la base de datos (PostgreSQL + PostGIS).

11. Conclusiones y Trabajo Futuro

11.1 Conclusiones

El desarrollo de Aupa ha demostrado la viabilidad de construir una plataforma de descubrimiento local end-to-end en el contexto de un bootcamp, integrando de forma coherente tecnologías modernas de frontend, backend y base de datos geoespacial.

Objetivos alcanzados:

- Motor de recomendaciones geoespacial basado en PostGIS.
- Sistema de autenticación completo con JWT y control de roles.
- Interfaz mobile-first responsiva con experiencia de usuario cuidada.
- Portal independiente para negocios locales (Local Partner).
- Panel de administración para la gestión global de la plataforma.
- Entorno reproducible y containerizado con Docker Compose.

11.2 Líneas de Trabajo Futuras

- Implementación de un motor de recomendaciones con aprendizaje automático (filtrado colaborativo) basado en el historial del usuario.
- Integración de notificaciones push para alertar de eventos próximos en la zona del usuario.
- Pasarela de pago (Stripe) para la reserva de experiencias de pago directamente desde la app.
- Ampliación del soporte de idiomas al euskera completo y al francés.
- Incorporación de tests automatizados de integración (pytest) y de componentes (Vitest + Testing Library).
- Migración del entorno Docker Compose a una plataforma cloud escalable (Railway, Fly.io o AWS ECS).